PYTHON FUNDAMENTALS

Lesson 1.1: Variable Declaration & Naming Conventions

Chapter 1: Variables and Primitive Data Types

LEARNING OBJECTIVES

- Understand Python's object reference model and dynamic typing.
- Use the syntax for assignment, reassignment and multiple assignment.
- Apply the PEP 8 naming conventions.
- Identify, fix and prevent common variable initialization and naming errors.

CORE CONCEPTS

Variables & Dynamic Memory Allocation

What is a Variable in Python?

- A variable is a name (label) that refers to an object stored in memory, not a fixed memory box.
- It is created automatically the first time you assign a value with the assignment operator (=).
- The name holds a reference to the object; the object holds the value and its type.

Dynamic Typing Mechanism

- **Types belong to objects, not variables:** no type declaration is needed before assignment.
- **Run-time typing:** the same name can later refer to an object of a different type.
- **Garbage collection:** objects that are no longer referenced are reclaimed automatically.

Memory Reference Inspection

```
# The name 'score' refers to an int object
score = 100
print(type(score), id(score))
# <class 'int'> 4370193552   (id varies)

# Rebinding 'score' to a new str object
score = "A+ Distinction"
print(type(score), id(score))
# <class 'str'> 4373841904   (id varies)
```

Key takeaway: reassignment does not change the old object. It binds the name to a different object; an object that nothing refers to any more can be reclaimed.

SYNTAX & EXAMPLES

Variable Declaration & Assignment Syntax

Fundamental Syntax: `variable_name = expression`

- **Left-hand side:** a valid identifier.
- **Assignment operator (=):** binds the name to the object produced by the right-hand side.
- **Right-hand side:** evaluated completely before the name is bound.

Supported Assignment Patterns

- **Standard assignment:** one name bound to the value of one expression.
- **Chained assignment:** several names bound to the same object. This is safe for immutable values; page 5 shows the pitfall with a mutable list.
- **Tuple unpacking:** several names assigned in one statement.
- **Swapping:** values exchanged without a temporary variable.

Code Demonstrations

```
# 1. Standard single assignment
course_name = "Python Fundamentals"
credit_hours = 3
passing_gpa = 2.5
is_published = True

# 2. Chained assignment (shared reference)
min_score = default_score = 0

# 3. Tuple unpacking / multiple assignment
x_coord, y_coord, z_coord = 10.5, -4.2, 0.0

# 4. Swapping two values
left_node, right_node = "Alpha", "Beta"
left_node, right_node = right_node, left_node
```

Identifier Rules & PEP 8 Naming Conventions

Mandatory Interpreter Rules (Syntax)

- **Allowed characters:** letters, digits (0–9) and underscores (_). Letters include non-ASCII Unicode letters, so `café` and `naïve` are valid names.
- **Leading character:** cannot be a digit (`1st_place` is invalid).
- **Case sensitivity:** `user_id`, `User_Id` and `USER_ID` are three distinct names.
- **No reserved keywords:** Python's 35 keywords (`if`, `for`, `class`, `def`, `return`, `None`, `global`, ...) cannot be names.

PEP 8 Style Guidelines (Best Practices)

- Use `snake_case` for variables and function names.
- Use `UPPER_SNAKE_CASE` for module-level constants.
- Prefer descriptive, intention-revealing names to single-letter abbreviations.
- Prefer ASCII letters (a–z, A–Z): they are easy to type and share, and PEP 8 requires them in the standard library.

Naming Comparison Matrix

Category	Valid & Pythonic	Invalid / Discouraged
Standard variable	`student_count`	`studentCount` camelCase
Boolean flag	`is_authenticated`	`flag1` vague meaning
Constant	`MAX_RETRY_LIMIT`	`max-retry` hyphen: syntax error
Internal / private	`_cache_buffer`	`2nd_buffer` starts with a digit
Reserved word	`class_room_id`	`class` SyntaxError

Tip: run `import keyword; print(keyword.kwlist)` to list all 35 reserved keywords.

TROUBLESHOOTING & PITFALLS

Common Variable Mistakes & How to Fix Them

Error type	Problematic code	Corrected code and resolution

1. Uninitialized variable
NameError
	`print(total_price)`
	The name is read before it is assigned, or it is misspelled.
		`total_price = 0.0`
		`print(total_price)`
		Assign a value before reading a name.

2. Shadowing a built-in
TypeError later
	`list = [10, 20, 30]`
	`new_list = list("abc")`
	The name `list` now refers to a list, not the built-in constructor.
		`scores_list = [10, 20, 30]`
		`chars = list("abc")`
		Avoid names such as list, str, int, id, sum and max.

3. Unpacking mismatch ValueError	`a, b = 10, 20, 30` Too many values to unpack: two names, three values.	`a, b, c = 10, 20, 30` or `a, *b = 10, 20, 30` Match the number of names, or use star unpacking.

4. Shared mutable reference
Logic bug
	`row_a = row_b = []`
	`row_a.append(1)`
	Both names refer to the same list, so row_b is also [1].
		`row_a, row_b = [], []`
		`row_a.append(1)`
		Create a separate mutable object for each name.

KNOWLEDGE CHECK

Mini Practice & Concept Checkpoint

Q1 (multiple choice): Which variable name is BOTH syntactically valid in Python and compliant with PEP 8?

A) `2nd_user_score`

B) `user-profile-id`

C) `max_connection_timeout`

D) `global`

KNOWLEDGE CHECK · ANSWER

Mini Practice & Concept Checkpoint

Q1 (multiple choice): Which variable name is BOTH syntactically valid in Python and compliant with PEP 8?

A) `2nd_user_score`

B) `user-profile-id`

C) `max_connection_timeout`

D) `global`

Answer: C. It uses only lowercase letters and underscores (snake_case). A starts with a digit, B contains hyphens and D is a reserved keyword.

KNOWLEDGE CHECK

Mini Practice & Concept Checkpoint

Q2: What is printed after this snippet runs?

```
x = 15; y = x; x = 40; print(y)
```

A) 40

B) 15

C) None

D) NameError

Mini Practice & Concept Checkpoint

Q2: What is printed after this snippet runs?

```
x = 15; y = x; x = 40; print(y)
```

A) 40

B) 15

C) None

D) NameError

Answer: B. y = x binds y to the object 15. Rebinding x to 40 afterwards does not change what y refers to.

KNOWLEDGE CHECK

Mini Practice & Concept Checkpoint

Buggy Code Snippet

```
# Fix 3 issues in this code:
1st_student = "Alice"
class = "PY101"
StudentAge = 20
```

Mini Practice & Concept Checkpoint

Suggested Fix

```
first_student = "Alice"   # a name cannot start with a digit
class_code = "PY101"      # class is a reserved keyword
student_age = 20          # PEP 8: snake_case for variables
```

The first two lines are syntax errors and stop the program from running. The third line runs, but it does not follow the PEP 8 naming convention.

SUMMARY

The End

Thank you

Primary Technical References

- Van Rossum, G., Warsaw, B., & Coghlan, N. (2001). *PEP 8 – Style Guide for Python Code*. Python Software Foundation. https://peps.python.org/pep-0008/
- Python Software Foundation. *The Python Language Reference*: Execution Model and Data Model. https://docs.python.org/3/reference/

Prepared for CodeMastery by the CodeMastery Project Team with AI assistance, for project teaching demonstrations, testing and academic evaluation.